

QUDA multigrid 算法实现

从近零模、Galerkin 粗化到 MPI 递归 V-cycle 的可复现逻辑

源码分析汇报

QUDA 1.1.0 参考源码快照

2026 年 8 月 28 日

本版报告的验收目标：读者可脱离 QUDA 重建同一算法

数学等价

给定同一细格算子 D 、零空间 B 和聚合规则，能够在 CPU、GPU 或任意语言中构造同一 P/R 、 $D_c = RDP$ 与 Schur 补系统。

控制流等价

伪代码明确写出 setup、batch、正交化、pre/post smooth、残差选择、递归、prepare/reconstruct 和停止条件。

实现可迁移

先给参考实现；再把 QUDA 的 checkerboard、halo、atomic、MMA、低精度和 cycle wrapper 映射回抽象接口。

边界声明

本报告确证源码控制流并给出可执行复现规范；本次未启动 CUDA/MPI，不虚报硬件收敛或性能。



来源：本报告结构依据源码：lib/multigrid.cpp:72-197, 1131-1224, 1275-1472；图为复现路线抽象。

先固定“算法合同”：只需要四类抽象接口

数据与线性代数

接口	必须提供的语义
Field	复数旋量/矩阵, 支持 full 与 parity subset
$D\psi$	细格 residual operator, 含边界条件与质量归一化
$\langle u, v \rangle$	全局或局部复内积; 范数为 $\sum u ^2$
P, R	同一聚合基上的 prolong/restrict

实现层必须保持的不变量

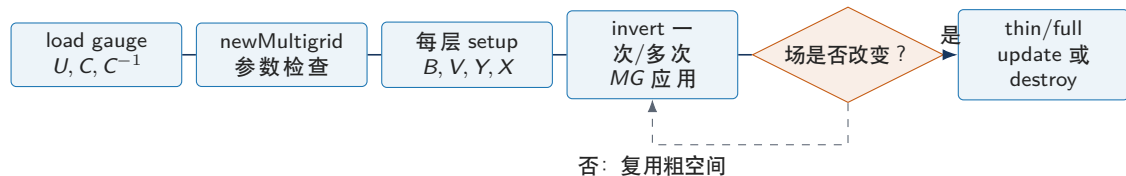
1. 粗点映射对每个细点唯一, 且 inverse map 覆盖整个聚合;
2. setup 中 (V) 的列在每个 aggregate 内正交归一;
3. $R = P^\dagger$ (本报告的 aggregate 路径);
4. coarse apply 与显式 (RDP) 的动作一致。

可移植策略

任何后端都可先用“逐粗基向量 apply $P \rightarrow D \rightarrow R$ ”构造 D_c , 通过验证后再替换为 UV/VUV 融合核。

来源: Transfer 接口: lib/transfer.cpp:259-328; 验证不变量: lib/multigrid.cpp:785-880, 883-1068。

总体执行链不是黑盒：API、setup、solve、update 各自有边界



构造时

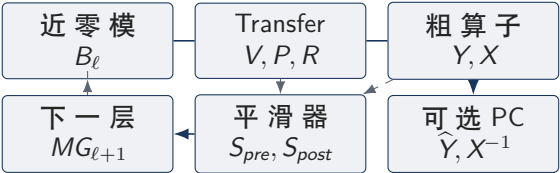
公共入口创建细格 residual/smoothen Dirac, 再以 (B) 创建 (MG(0)); 非粗层由 reset() 创建 Transfer、粗场、粗 Dirac、下一层与 coarse solver。

更新时

thin update 只更新 Dirac 所引用的 gauge/clover 与质量; full update 重新创建细格 Dirac 并递归 reset(refresh=true), 必要时刷新零空间。

来源：公共入口：lib/interface_quda.cpp:2853-2944; 更新：lib/interface_quda.cpp:2946-3053; 递归 reset：lib/multigrid.cpp:72-197。

每个非底层 MG_ℓ 的状态：六个对象足以复现运行期



状态	用途
B_ℓ	setup 的输入/输出，下一层的候选零空间
V_ℓ	aggregate-local 的列正交基
Y, X	粗 hopping 与 local term
$Yhat, X^{-1}$	even/odd 预条件 dslash
smoother	负责局部高频误差
coarse solver	递归 V-cycle 或 底层 Krylov

复现顺序

先实现 $B \rightarrow V \rightarrow P/R \rightarrow D_c$ ；再实现 $X^{-1}, Yhat$ ；最后接入 smooth/recursive solve。每一步都能单独与显式矩阵动作对比。

维数协议 (1/2): 先缩小时空, 再重组自旋与颜色自由度

$$\begin{aligned}L_c^\mu &= L_f^\mu / b_\mu, & N_c^{\text{color}} &= N_{\text{vec}}, \\N_c^{\text{spin}} &= N_f^{\text{spin}} / b_s & & (b_s > 0), \\N_c^{\text{spin}} &= 2 & & (b_s = 0, \text{ staggered}).\end{aligned}$$

场的抽象形状

细场 B_i : $L_f^4 \times N_f^s \times N_f^c$; 粗场: $L_c^4 \times N_c^s \times N_c^c$.
 V 的每个 coarse color 是一列局部基。

存储边界

Y 是每个粗点的 4/8 个方向块, 通常 nFace=1;
 X 是每个粗点的 local 方阵, 无 ghost。
checkerboard 只改变线性索引, 不改变这些矩阵形状。

来源: 粗维数与 (Y/X) 创建: lib/dirac_coarse.cpp:121-170; 布局解释为可迁移抽象。

维数协议 (2/2): aggregate 容量是 N_{vec} 的硬上限

$$C_{\text{agg}} = \left(\prod_{\mu=0}^3 b_{\mu} \right) N_f^{\text{color}} \times \begin{cases} b_s, & b_s > 0, \\ 1/2, & b_s = 0 \text{ (staggered)}, \end{cases} \quad N_{\text{vec}} \leq C_{\text{agg}}.$$

为什么是上限

C_{agg} 是一个 aggregate 内可提供的独立局部自由度。超过它时，块 Gram-Schmidt 必然产生零列或线性相关列，后续 P/R 不可能形成单位局部基。

来源：容量与块大小上限：lib/transfer.cpp:25-83、lib/block_orthogonalize.in.cu:85-96。

先验检查

同时检查 $1 < \prod b_{\mu} \leq 1024$ 、粗体积按 parity 可分和每维整除；不要只检查总 lattice volume。

奇偶与线性索引 (1/3): 构造 fine-to-coarse map

$$p(x) = \left(\sum_{\mu=0}^3 x_{\mu} \right) \bmod 2, \quad x_c^{\mu} = \left\lfloor \frac{x_f^{\mu}}{b_{\mu}} \right\rfloor.$$

正向 map

`fine_to_coarse[i]` 把细格
parity-ordered 索引 i 映射到粗点
 k_c 。

必须记录

保留 x_f 、 p_f 、 x_c 三者，不要只存扁平 k_c ； P/R 还要用原坐标恢复 neighbor。

来源：正向 map: `lib/transfer.cpp:200-241`。伪代码为等价抽象。
源码分析汇报 QUDA multigrid 复现手册

```
1 for each fine linear index i:
2     x_f, p_f = lattice_coordinate_and_parity(i)
3     x_c = floor_div(x_f, geo_block_size)
4     k_c = parity_ordered_index(x_c, parity(x_c))
5     fine_to_coarse[i] = k_c
6
7 for each i:
8     pairs.append((fine_to_coarse[i], i))
9 sort(pairs) # group by coarse
10 site
coarse_to_fine = [pair.fine_index for pair in pairs]
```


奇偶与线性索引 (2/3): 构造 coarse-to-fine 逆向列表

```
1 for each i:  
2     pairs.append((fine_to_coarse[i], i))  
3 sort(pairs)                                # group by coarse site  
4 coarse_to_fine = [pair.fine_index for pair in pairs]  
5  
6 for each coarse site q and parity p:  
7     assert count({i : fine_to_coarse[i] == q  
8                   and parity(i) == p}) > 0  
9
```

连续性

逆向表按 (k_c, i_f) 排序, 使同一 aggregate 的细点连续; GPU kernel 才能用一个 thread block 处理同一块。

来源: 逆向 map: lib/transfer.cpp:200-241; kernel 读取: include/kernels/coarse_op_kernel.cuh:1693-1704。
伪代码为等价抽象。

不变量

每个细点出现一次; 每个粗点的列表覆盖所属细点, 不能遗漏、重复或交换 parity。

奇偶与线性索引 (3/3): spin map 必须和 P/R 共用同一 parity

```
1 if spin_block_size == 0:           # staggered
2     spin_map[0][even] = 0
3     spin_map[0][odd]  = 1
4 else:
5     for s_f in fine_spin:
6         for p in {even, odd}:
7             spin_map[s_f][p] = floor_div(s_f,
              spin_block_size)
```

读取时的索引链

先由 x_f 查 k_c , 再由 p_f, s_f 查 s_c 。同一个 spin_map 必须被块正交化、 P 、 R 和粗算子 setup 复用。

实现检查

对每个 coarse 点, coarse_to_fine 必须列出属于它的细点; 对每个 parity subset, spin map 不能切换到另一套编号。

```
1 for each coarse site q and parity p:
2     assert count({i : fine_to_coarse[i] == q and parity(i) == p}) > 0
3     assert all(P/R use the same spin_map[s_f][p])
4
```

来源: spin map: lib/transfer.cpp:243-257; P/R parity subset: lib/transfer.cpp:259-328。伪代码为等价抽象。

单层 setup 总览 (1/2): 先固定几何、零空间和 transfer

```
1 setup_level(level, D, U, C, B, cfg):
2     validate_geometry_and_capacity(U.shape, cfg.geo_block_size,
3                                     cfg.spin_block_size, cfg.n_vec)
4
5     if cfg.compute_null:
6         B = initialize_or_load_or_generate(B, cfg.null_source)
7         B = generate_null_vectors(B, level, cfg)
8
9     geo_map = build_geometry_maps(U.shape, cfg.geo_block_size)
10    spin_map = build_spin_map(U.n_spin, cfg.spin_block_size)
11    V = block_orthonormalize(B, geo_map, spin_map, cfg)
12    P = make_prolongator(V, geo_map, spin_map)
13    R = make_restructor(V, geo_map, spin_map)      # R = P^dagger
14    return SetupState(B, V, P, R, geo_map, spin_map)
```

此阶段的闭环

B 决定 V , V 决定 P/R 。先通过 map 覆盖检查与 $R = P^\dagger$, 再进入粗算子装配。

不可交换的顺序

不能先分配 coarse Dirac 再猜 V 的布局;
粗自旋、粗颜色、parity 和 ghost 元数据都由本阶段确定。

单层 setup 总览 (2/2): 由 transfer 装配粗算子并创建下一层

```
1 finish_level(state, D, U, C, cfg):
2     (B, V, P, R, geo_map, spin_map) = state
3     (Y, X) = build_galerkin_operator(U, C, V, geo_map, spin_map, cfg)
4
5     Xinv = None; Yhat = None
6     if cfg.needs_pc:
7         Xinv = inverse_local_blocks(X)
8         Yhat = build_preconditioned_links(Y, Xinv)
9
10    coarse = allocate_next_level(coarse_geometry(U.shape, geo_map),
11                                Nvec=cfg.n_vec, Nspin=coarse_spin)
12    verify_transfer_and_operator(P, R, D, coarse_apply(Y, X), cfg)
13    return Level(B, V, P, R, Y, X, Xinv, Yhat, coarse)
14
```

依赖链

P/R 确定 $D_c = RDP$; Y/X 确定 coarse Dirac; $X^{-1}, \hat{Y}$ 再确定 PC Schur 链。

可替换点

build_galerkin_operator 可用显式 basis apply; 通过 $D_c - RDP$ 后, 再替换为 QUDA 的 $UV \rightarrow VUV$ 计算图。

几何检查 (1/2): 非法 block 的逐维减半行为

```
1 for d in dimensions 0..3:
2     while b[d] > 0:
3         coarse_d = L_f[d] / b[d]
4         invalid = (d == 0 and L_f[0] == b[0])
5                 or ((coarse_d + 1) % 2 == 0)
6                 or (coarse_d * b[d] != L_f[d])
7         if not invalid:
8             break
9         warn_or_record(d, b[d])
10        b[d] = b[d] / 2
11    if b[d] == 0:
12        fail("dimension cannot be blocked")
13
```

判定含义

每一维都要求整除；粗维还必须能按 parity 分开。遇到非法值时记录 warning 并减半，直到可用或失败。

两种复现目标

数学复现可直接拒绝非法配置；行为复现则必须保留逐维减半、warning 和最终 block。

来源：检查与减半：lib/transfer.cpp:25-83；粗维 parity 约束：lib/transfer.cpp:45-83。伪代码显式保留行为分支。

几何检查 (2/2): aggregate 容量决定 N_{vec} 能否存在

```
1 aggregate = product(b)
2 if spin_block == 0:                # staggered
3     capacity = aggregate * N_f_color / 2
4 else:
5     capacity = aggregate * N_f_color * spin_block
6
7 assert Nvec <= capacity
8 assert 1 < aggregate <= 1024
9 assert aggregate % 2 == 0
10
```

物理/线性代数含义

一个 aggregate 能提供的独立局部自由度就是粗基列数的上限；若 $N_{\text{vec}} > C_{\text{agg}}$ ，局部 Gram-Schmidt 必然出现零向量。

来源：容量与块大小上限：lib/transfer.cpp:25-83、lib/block_orthogonalize.in.cu:85-96。

实现含义

C_{agg} 与 b_μ 、细颜色、spin block 共同决定粗场维数；不要只检查总 lattice volume。

近零模生成 (1/3): 先确定 setup solver 的数值语义

```
1 configure_null_setup(level, cfg, refresh=false):  
2     sp.maxiter = refresh ? cfg.setup_maxiter_refresh[level]  
3                   : cfg.setup_maxiter[level]  
4     sp.tol = cfg.setup_tol[level]  
5     sp.use_init_guess = YES  
6     sp.delta = 1e-1  
7     sp.inv_type = cfg.setup_inv_type[level]  
8     sp.precision = cfg.cuda_prec_sloppy  
9     sp.residual_type = L2_RELATIVE  
10    sp.compute_null_vector = YES  
11    return sp  
12
```

为什么是“近零模”

setup solver 反复削弱 D 的高频分量, 使 B_i 落入低频子空间; 它不是任意随机向量的存档。

来源: setup solver 参数: lib/multigrid.cpp:1275-1363; 参数来源: tests/utils/set_params.cpp:516-582。伪代码剖析抽象。

必须保持的状态

use_init_guess = YES、相对残差类型、sloppy 精度和 refresh 的独立 maxiter 都会改变最终 B , 不能只复现 solver 名称。

近零模生成 (2/3): 分批 solve 并把结果写回同一组 B_i

```
1 generate_null_vectors(B, level, cfg, refresh=false):
2     sp = configure_null_setup(level, cfg, refresh)
3     batch = cfg.n_vec_batch[level]
4     require len(B) % batch == 0
5
6     for setup_iter in 1 .. cfg.num_setup_iter[level]:
7         if cfg.pre_orthonormalize:
8             global_gram_schmidt(B)
9         for first in range(0, len(B), batch):
10             B_i = B[first:first+batch]
11             if cfg.setup_type == TEST_VECTOR_SETUP:
12                 rhs = copy(B_i); x = zero_like(B_i)
13             else:
14                 x = copy(B_i); rhs = zero_like(B_i)
15             solve_setup_system(x, rhs, sp)
16             B[first:first+batch] = x
17         if cfg.post_orthonormalize:
18             global_gram_schmidt(B)
19         refresh_transfer_and_next_level_if_needed(B, cfg)
20     return B
21
```


近零模生成 (2/2): CG、GCR、MG 三条内部求解路径不能混写

CG / CA-CG

setup solver 不是直接作用于 D , 而是包装 DiracMdagM, 即以 $D^\dagger D$ 为求解矩阵。
CA-CG 另外设置 Chebyshev/CA basis 参数。

普通 Krylov

GCR、BiCGStab-L 等由 Solver::create 直接以 matSmooth 为矩阵; 若输入配置带 Schwarz, 源码在此阶段先暂时关闭它。

MG setup 自举

当 setup_inv_type=MG:

setup solver = GCR, $K = MG_\ell$, 每轮最多

先保证本层 smoother 存在, 再用本层 MG 作为 GCR 的 preconditioner。

递归更新

generate_all_levels=true: 向下层重新生成; 否则先 $B_{\ell+1} = R_\ell B_\ell$, 再重建下一层 Transfer。自举完成后恢复 Schwarz/halo 精度并释放临时 solver。

来源: solver 分支: lib/multigrid.cpp:1335-1363; MG 自举与向下传播: lib/multigrid.cpp:1429-1466; Schwarz 临时关闭: lib/multigrid.cpp:1321-1333。
源码分析汇报

全局正交化与块正交化是两层不同约束

全局正交化（可选）

作用域是整个 local/global lattice 的 B_i :

$$B_i \leftarrow B_i - \sum_{j < i} \langle B_j, B_i \rangle B_j, \quad B_i \leftarrow B_i / \|B_i\|.$$

由 `pre_orthonormalize`、`post_orthonormalize` 控制，内积可能跨 MPI reduction。

块正交化（Transfer 构造必需）

每个 aggregate、每个 coarse-spin/chirality block 单独执行 Gram-Schmidt；它保证的是 $V^\dagger V \approx I$ 的局部关系，而不是不同 aggregate 之间的全局正交。

全局 B_i : $\langle B_j, B_i \rangle$

按 aggregate
聚类细点

局部 V_i : 内
积与归一

不要用全局 $B^\dagger B = I$ 替代块正交

Galerkin transfer 的局部列基按 aggregate 存储；只做全局正交并不能使同一 aggregate 内的列成为 $P^\dagger P$ 的单位块。

来源：全局正交化：lib/multigrid.cpp:1369-1381, 1415-1427；块正交化：

include/kernels/block_orthogonalize.cuh:141-253。

QUADA multigrid 复现手册

块正交化伪代码 (1/3): 按 aggregate 和向量批次装载候选列

```
1 block_orthonormalize(B, geo_map, spin_map, n_block_ortho):
2   V = allocate_matrix_field(n_vec, fine_spin, fine_color)
3   for each coarse site q:
4     for each coarse chirality h:
5       for repeat in 0 .. n_block_ortho-1:
6         for j0 in 0, mVec, 2*mVec, ... n_vec-mVec:
7           v[0:mVec] = load(B if repeat == 0 else V,
8                           q, aggregate(q), h, j0:j0+mVec)
9         for i in 0 .. j0-1:
10          vi = load(V, q, aggregate(q), h, i)
11          for m in 0 .. mVec-1:
12            coeff[m] = sum_{x in aggregate(q)}
13                      inner(vi(x), v[m](x))
14          coeff = reduce_within_aggregate(coeff)
15          for m in 0 .. mVec-1:
16            v[m] = v[m] - coeff[m] * vi
17          local_block_orthogonalize_and_normalize(v)
18          store(V, q, aggregate(q), h, j0:j0+mVec, v)
19
```

块正交化伪代码 (2/3): tile 内的列间正交和归一化

```
1 local_block_orthogonalize_and_normalize(v[0:mVec]):  
2     for m in 0 .. mVec-1:  
3         for i in 0 .. m-1:  
4             c = reduce_{x in aggregate}(inner(v[i](x), v[m](x)))  
5             v[m] = v[m] - c * v[i]  
6         n2 = reduce_{x in aggregate}(norm2(v[m](x)))  
7         if n2 <= 0: fail("zero null vector in aggregate")  
8         v[m] = v[m] / sqrt(n2)  
9
```

两类正交项

外层 $i < j_0$ 处理当前 tile 与历史列; tile 内 $i < m$ 处理同批列之间的交叉项。少掉任一层都会使 $V^\dagger V$ 偏离单位阵。

归一化边界

n^2 是 aggregate 内的全局 (或 rank-local 后再 reduction) 范数; $n^2 \leq 0$ 必须报告为零基向量, 不能静默除零。

来源: tile 内正交与归一化: include/kernels/block_orthogonalize.cuh:215-251。

块正交化伪代码 (3/3): 重复正交与低精度 two-pass

```
1 two_pass_block_ortho(B, cfg):
2     V = block_orthonormalize(B, ..., cfg.n_block_ortho)
3     if cfg.block_ortho_two_pass and V.precision < SINGLE:
4         scale = abs_max(V)
5         if 1.05 * scale < 1:
6             V.set_storage_scale(1.05 * scale)
7             V = block_orthonormalize(B, ..., cfg.n_block_ortho)
8     return V
9
```

重复次数

`n_block_ortho` 重复整套块 Gram-Schmidt; 它抑制舍入残差, 但不是改变数学定义的额外基。

two-pass 的边界

先用实际最大幅值设置 fixed-point/half scale, 再从原始 B 重算; 这是存储精度补救, 不是额外的收敛判据, scale 必须随 V 保存。

P/R 的数学定义 (1/2): prolongation 是按局部基展开

$$(P\psi_c)_{s_f c_f}(x_f) = \sum_{a=0}^{N_c^c-1} V_{s_f c_f, a}(x_f) \psi_{c, s_c(s_f, p_f), a}(x_c(x_f)).$$

局部基的角色

$V(x_f)$ 的列把 coarse color a 旋转到细 spin/color; $x_c(x_f)$ 和 $s_c(s_f, p_f)$ 只负责位置与 coarse-spin 映射。

单 parity

full field 计算两个 parity; single-parity 由 `Transfer::setSiteSubset` 固定目标 parity, 不能把单 parity 的 V 当作 full-field 基。

来源: P 入口: `lib/transfer.cpp:259-292`; kernel 旋转: `include/kernels/prolongator.cuh:65-132`。

P/R 的数学定义 (2/2): restriction 是共轭转置和 aggregate reduction

$$(R\psi_f)_{s_c a}(x_c) = \sum_{x_f \in A(x_c)} \sum_{s_f, c_f} V_{s_f c_f, a}^*(x_f) \psi_{f, s_f c_f}(x_f).$$

局部单位性

若每个 aggregate 内 $V^\dagger V = I$, 则

$$(RP\eta_c)(x_c) = \eta_c(x_c).$$

这是 coarse span 检查的理论基准; 它同时要求 map、清零、累加和共轭方向全部正确。



P 参考实现：先保证位置映射、清零和 parity 完全正确

```
1 prolongate(out_f, in_c):  
2     for each fine site x_f in requested parity subset:  
3         q = geo_map[x_f]; p = parity(x_f)  
4         for s_f, c_f:  
5             s_c = spin_map[s_f][p]  
6             out_f[x_f,s_f,c_f] = 0  
7             for a in coarse_colors:  
8                 out_f[x_f,s_f,c_f] += V[x_f,s_f,c_f,a] \  
9                                         * in_c[q,s_c,a]  
10
```

GPU 映射

一个线程可负责细点/粗色块；一个 block 可负责一个 aggregate；MMA 只替换内层矩阵乘。

来源：Prolongate: include/kernels/prolongator.cuh:65-132。伪代码为等价实现。

不变量

每个细点先清零再对 coarse color a 求和； q, p, s_c 必须来自同一组 map。

R 参考实现：清零后做共轭基累加和 reduction

```
1 restrict(out_c, in_f):  
2     fill(out_c, 0)  
3     for each fine site x_f in requested parity subset:  
4         q = geo_map[x_f]; p = parity(x_f)  
5         for s_f, c_f, a:  
6             s_c = spin_map[s_f][p]  
7             out_c[q,s_c,a] += conj(V[x_f,s_f,c_f,a]) \  
8                                 * in_f[x_f,s_f,c_f]  
9     reduce_out_c_across_threads_and_ranks(out_c)  
10    return out_c  
11
```

第一条单元测试

随机粗场检查 $\|R(P\eta_c) - \eta_c\|/\|\eta_c\|$ ；随机细场检查 R 与显式共轭转置的差异。若输出未清零，误差会随重复调用累积。

来源：Restrict 的 zero/reduction: lib/transfer.cpp:292-328、include/kernels/restrictor.cuh:126-150。伪代码为等价实现。

Galerkin golden reference (1/2): 逐粗基列执行 $P \rightarrow D \rightarrow R$

```
1 build_galerkin_reference(D, P, R, coarse_shape):  
2     Dc = zero_matrix_or_operator(coarse_shape)  
3     for each coarse source basis index beta=(q,s_c,a):  
4         e_c = unit_vector(beta)  
5         e_f = P(e_c)  
6         y_f = D(e_f)  
7         column = R(y_f)  
8         Dc[:, beta] = column  
9     return Dc  
10
```

为什么可迁移

只依赖 D, P, R 的动作, 不依赖 CUDA thread/block、GaugeField order 或 atomic; 复杂度大, 但可作为 CPU golden reference。

来源: Galerkin verify 的比较对象: lib/multigrid.cpp:883-928。伪代码定义了可移植 golden path。

列/行约定

e_c 是 coarse 的单位列; $R(D(Pe_c))$ 直接写入 D_c 的同一列。若转置或共轭方向错, $D_c - RDP$ 会立即暴露。

Galerkin golden reference (2/2): 把显式 D_c 拆成 Y 与 X

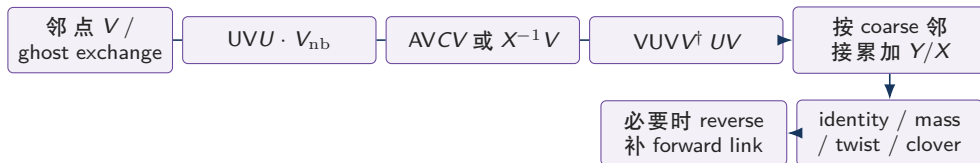
```
1 split_into_Y_and_X(Dc):
2     for each output q and input q2:
3         if q2 == q:
4             X[q] += Dc[q,q2]
5         elif q2 == q + unit_direction(mu):
6             Y_fwd[mu,q] = Dc[q,q2]
7         elif q2 == q - unit_direction(mu):
8             Y_back[mu,q] = Dc[q,q2]
9         else:
10             preserve_explicit_long_range_term_or_fail()
11     return Y_fwd, Y_back, X
12
```

$$D_c = RDP, \quad Y_{q \rightarrow q'} = \sum_{x \in A_q, y \in A_{q'}} V^\dagger(x) H(x, y) V(y), \quad X_q = \sum_{x, y \in A_q} V^\dagger(x) D(x, y) V(y).$$

结构约束

标准 Wilson/Clover aggregate 把跨 coarse-site 的一跳项放进 Y , 同 aggregate 内项放进 X 。含长链时必须显式决定保留长链接, 或按 QUDA 的 truncation 规则拒绝/截断。

QUDA 的快速 Galerkin 图 (1/2): UV/VUV 是 golden path 的融合与复用



正确性关系

UV/VUV 只把 RDP 中可复用的邻点乘法和局部矩阵乘法融合；它必须与 golden reference 的 D_c 在每个 coarse matrix element 上一致。

来源: UV/VUV 调度: lib/coarse_op.cuh:1043-1062, 1166-1215, 1269-1323; VUV kernel: include/kernels/coarse_op_kernel.cuh:1067-1138。

QUDA 的快速 Galerkin 图 (2/2): projector、系数与存储边界

$$D_c = RDP, \quad H_{+\mu} = (1 - \gamma_\mu)U_\mu, \quad H_{-\mu} = (1 + \gamma_\mu)U_\mu^\dagger.$$

Wilson/Clover

forward 使用 $1 - \gamma_\mu$, backward 使用 $1 + \gamma_\mu$; spin 对角和非对角项分别累加到 coarse spin block。

来源: Wilson projector: `include/kernels/coarse_op_kernel.cuh:1067-1138`; 应用时 $-\kappa$: `include/kernels/dslash_coarse.cuh:292-324`。

κ 与 X

外部 Y 通常不含应用时的 $-\kappa$; 粗 dslash 汇总后再乘 $-\kappa$ 。内部项已在写入 X 前乘相应系数。

Galerkin 的关键分类 (1/2): fine hopping 决定写 Y 还是 X

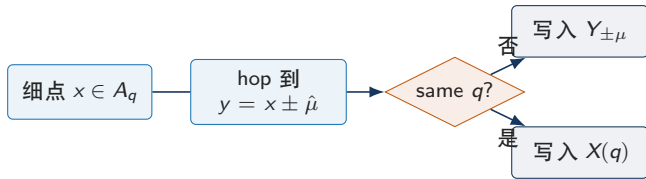
$$H_{+\mu}(x, y) = (1 - \gamma_\mu) U_\mu(x) \delta_{y, x+\hat{\mu}}, \quad H_{-\mu}(x, y) = (1 + \gamma_\mu) U_\mu^\dagger(x - \hat{\mu}) \delta_{y, x-\hat{\mu}}.$$

同一粗点

若 $\lfloor x/b \rfloor = \lfloor (x \pm \hat{\mu})/b \rfloor$, 该项是 local term, 累加 $X(q)$, 并保留完整 spin/color 矩阵。

相邻粗点

若终点落入 $q \pm \hat{\mu}$, 该项写入对应方向的 Y ; apply 时从 neighbor coarse spinor gather。



来源: 对角判定与写回: include/kernels/coarse_op_kernel.cuh:1548-1647。图为同一判定的可执行解释。

Galerkin 的关键分类 (2/2): 长链接与方向存储不可含糊

一跳 Wilson/Clover

$Y_{+\mu}(q)$ 表示从 q gather $q + \hat{\mu}$ 的 forward block; $Y_{-\mu}(q)$ 表示从 q gather $q - \hat{\mu}$ 的 backward block。方向、坐标和 dagger 必须和 coarse dslash 的 gather 完全一致。

长链接

ASQTAD/HISQ 需要 $nFace = 3$ 。若聚合方向小于 3, 不能假设一跳 Y 足够; 按 `allow_truncation` 选择截断、保留长链接或拒绝配置。

互斥分类

每个 fine hopping 恰好进入 X 、某个 $Y_{\pm\mu}$ 或明确的 long-range bucket; 重复计数和静默丢项都会破坏 $D_c - RDP$ 。

$$D_c(q, q') = \begin{cases} X(q), & q' = q, \\ Y_{+\mu}(q), & q' = q + \hat{\mu}, \\ Y_{-\mu}(q), & q' = q - \hat{\mu}. \end{cases}$$

X 的组成 (1/2): 内部 hopping 与 clover 投影

$$X(q) = X_{\text{internal hop}}(q) + X_{\text{clover}}(q) + X_{\text{local diag}}(q).$$

分量	QUDA 逻辑与不可重复计算的边界
internal hop	$V^\dagger HV$, 写入 X , 已带 hopping 系数; 只接收 aggregate 内部项
clover	fine Clover 的 $V^\dagger CV$; 仅在选择 clover 构造时加入

与 Y 的互斥性

终点仍在 A_q 的 fine hopping 才进入 $X(q)$; 终点落入邻 aggregate 的项必须进入 $Y_{\pm\mu}$, 不能在两个位置重复累加。

来源: internal/clover: lib/coarse_op.cuh:1383-1441; 对角/非对角分支:
include/kernels/coarse_op_kernel.cuh:1548-1647。

X 的组成 (2/2): identity、mass、twisted-mass 逐项补齐

$$X_{\text{local diag}}(q) = l_{\text{id}} + 2m l_{\text{stag}} + i\mu \text{diag}(+1_{\text{upper}}, -1_{\text{lower}}).$$

分量	触发条件与实现语义
identity	非 staggered 路径显式加 coarse identity
staggered mass	staggered 路径对角加 $2ml$
twisted mass	coarse spin 上半块加 $+i\mu$, 下半块加 $-i\mu$

装配边界

COMPUTE_COARSE_CLOVER 只构造 fine clover 投影; internal hopping 已在 VUV diagonal 分支进入 X, 不能再次加。

来源: identity/mass/twist: lib/coarse_op.cuh:1383-1441; 对应 kernel:
include/kernels/coarse_op_kernel.cuh:1768-1950。

X 的装配伪代码 (1/2): 逐 fine hopping 判断是否留在 aggregate

```
1 build_local_hop_block(V, fine_operator, aggregate q):
2     X = zero_matrix(coarse_dof, coarse_dof)
3     for x in sites(q):
4         for mu in 0 .. 3:
5             y = neighbor(x, +mu)
6             if aggregate_id(y) == q:
7                 H = forward_hop(fine_operator, x, mu)
8                 X += dagger(V[x]) * H * V[y]
9             y = neighbor(x, -mu)
10            if aggregate_id(y) == q:
11                H = backward_hop(fine_operator, x, mu)
12                X += dagger(V[x]) * H * V[y]
13    return X
```

关键不变量

此函数只接收 aggregate 内部 hopping; 跨 aggregate 的项由同一套 $V^\dagger H V$ 累加到 $Y_{\pm\mu}$ 。

来源: 对角/非对角分支: `include/kernels/coarse_op_kernel.cuh:1548-1647`。伪代码为可移植等价实现。

X 的装配伪代码 (2/2): 在 hopping 后按算子类型补 local term

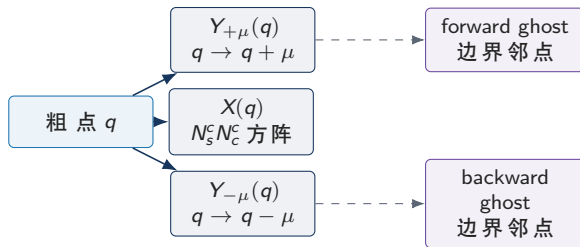
```
1 finish_local_block(X, V, fine_operator, aggregate q, cfg):  
2     if cfg.has_clover:  
3         for x in sites(q):  
4             X += dagger(V[x]) * clover(x) * V[x]  
5     else if not cfg.is_staggered:  
6         X += identity_matrix()  
7     if cfg.is_staggered:  
8         X += 2 * cfg.mass * identity_matrix()  
9     if cfg.is_twisted:  
10        add_twisted_spin_diagonal(X, cfg.mu, cfg.spin_sign)  
11    return X
```

执行顺序

先完成 $V^\dagger H V$ 的 internal 分支, 再执行 clover/identity/mass/twist 分支; 每一项只执行一次, 并在写回前固定精度与 scale。

来源: local 补项: lib/coarse_op.cuh:1383-1441; kernel: include/kernels/coarse_op_kernel.cuh:1768-1950。

粗场存储协议: (Y) 有方向和 ghost, (X) 是每点 local block



代码创建的元数据

粗 (Y): full site subset、coarse geometry、nFace=1、双向 link ghost; 粗 (X): scalar geometry、nFace=0、无 ghost。GPU 可能另存 MILC/AoS copy 供 MMA。

方向索引

在粗 dslash 中, forward gather 使用 $Y[d+4](q)$, backward gather 使用 $Y[d](q - \hat{d})^\dagger$ 。

来源: 字段创建: lib/dirac_coarse.cpp:121-170; 方向 gather: include/kernels/dslash_coarse.cuh:150-249。

PC 粗化 (1/2): 先求 X^{-1} , 再构造 $\hat{Y}$ 的左右乘法

$$X^{-1}(q)X(q) = I, \quad \begin{aligned} Yhat_{+\mu}(q) &= X^{-1}(q)Y_{+\mu}(q), \\ Yhat_{-\mu}(q - \hat{\mu}) &= Y_{-\mu}(q - \hat{\mu})X^{-\dagger}(q). \end{aligned}$$

```
1 build_pc_links(Y, X):
2     Xinv[q] = batch_inverse(X[q])          # local, independent for each q
3     exchange_ghost(Y, both_directions)
4     for direction mu and coarse site q:
5         Yhat_fwd[q,mu] = Xinv[q] * Y_fwd[q,mu]
6         # backward link is stored at q-mu; its right factor is Xinv(q)^dagger
7         Yhat_back[q-mu,mu] = Y_back[q-mu,mu] * dagger(Xinv[q])
8     exchange_ghost(Yhat, both_directions)
9     return Xinv, Yhat
```

最容易写反的地方

forward 是左乘 X^{-1} , backward 是右乘 $X^{-\dagger}$, 且 backward link 的坐标是邻点 $q - \mu$ 。逐点写成统一的 $X^{-1}Y$ 会破坏 $\hat{D} - A^{-1}D$ 检查。

PC 粗化 (2/2): Schur 补与 QUDA 的 DiracCoarsePC::M 对齐

$$D = \begin{pmatrix} A_{ee} & D_{eo} \\ D_{oe} & A_{oo} \end{pmatrix}, \quad S_e = A_{ee} - D_{eo} A_{oo}^{-1} D_{oe}$$

对称 PC

输入只保留目标 parity; Dslash 负责 $A^{-1}D$ 的 neighbor hop, DslashXpay(..., -1) 把两个 hopping 串联并减去单位项, 得到目标 parity 的 Schur 动作。

```
1 apply_schur(in_target, target_parity):
2     other = 1 - target_parity
3     tmp_other = A_other^{-1} * D_other, target *
4         in_target
5     tmp_target = A_target^{-1} * D_target, other *
6         tmp_other
7
8     out = tmp_target - in_target          # QUDA
9     DslashXpay k=-1
10    return out
11
12 prepare_full_rhs(b_even, b_odd):
13
14    src = A_other^{-1} * b_other
15    src = A_target^{-1} * b_target -
16        A_target^{-1} * D_target, other * src
17    return src
18
19 reconstruct_full(x_target, b_other):
20
21    x_other = A_other^{-1} * b_other -
22        A_other^{-1} * D_other, target *
23        x_target
24    return (x_target, x_other)
```

粗 dslash 的单点动作：先 forward/backward gather，再统一乘 $(-\kappa)$

$$(D_{\text{hop}}\psi)(q) = \sum_{\mu=0}^3 \left[Y_{+\mu}(q)\psi(q + \hat{\mu}) + Y_{-\mu}^{\dagger}(q - \hat{\mu})\psi(q - \hat{\mu}) \right],$$

```
1 coarse_dslash(out, in, parity, use_X):
2     for q in destination_sites(parity):
3         acc = zero_vector(q)
4         for mu in 0..3:
5             qf = neighbor(q, +mu)
6             qb = neighbor(q, -mu)
7             psi_f = halo_or_local(in, qf, parity^1)
8             psi_b = halo_or_local(in, qb, parity^1)
9             acc += Y_fwd[q,mu] * psi_f
10            acc += dagger(Y_back[qb,mu]) * psi_b
11        acc *= -kappa
12        if use_X:
13            acc += X[q] * in[q, parity]
14        out[q, parity] = acc
15
```

MPI/halo 复现 (1/2): 通信只改变 neighbor 数据的取得方式

```
1 exchange_spinor_halo(in, depth=1):
2     for mu in dimensions:
3         pack plus_face = in[coord[mu] ==
4             L_local[mu]-1]
5         pack minus_face = in[coord[mu] == 0]
6         include parity, spin, color, rhs_batch
7         post Irecv(plus_ghost <- rank_minus_mu)
8         post Irecv(minus_ghost <- rank_plus_mu)
9         post Isend(plus_face -> rank_plus_mu)
10        post Isend(minus_face -> rank_minus_mu)
11    return requests, ghost_buffers
12
13 apply_coarse_dslash:
14     requests, ghost = exchange_spinor_halo(in)
15     compute_interior_sites(out, in)      #
16     overlap
17     wait_all(requests)
18     compute_boundary_sites(out, in, ghost)
```

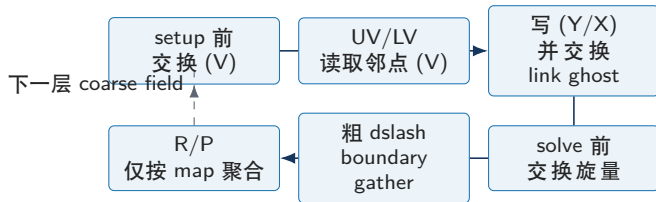
每个粗 dslash 只需一层

Wilson/Clover coarse hopping 是一跳；因此 $nFace = 1$ 。ASQTAD 细格 setup 的 UV/LV 需要 $nFace=3$ ，不要把这个 setup halo 深度误用于粗 dslash。

单 rank 极限

$rank_plus_mu=rank_minus_mu=self$ 时用周期坐标直接索引即可；这应与 MPI 版本共享同一 `neighbor()` 逻辑。

MPI/halo 复现 (2/2): 链接 halo、旋量 halo 和聚合边界要分开



setup 期间

calculateY 先让 V 交换 ghost; 跨 rank 的 V 才能得到正确边界 coarse link。写好 Y 后, PC calculateYhat 还要让 Y 和 $\hat{Y}$ 的双向 link ghost 有效。

aggregate 不跨 rank 的假设不能擅自删除

map 是 local volume 上的聚合索引; 若 block 跨 MPI 分区, 需要先重新定义 aggregate ownership 或通信, 不能仅靠 coarse dslash 的 halo 修补 setup 的 V 访问。

来源: V ghost: lib/coarse_op.cuh:1056-1062; Y link ghost: lib/coarse_op_preconditioned.in.cu:197-203、lib/dirac_coarse.cpp:260-270; 粗 dslash boundary: include/kernels/dslash_coarse.cuh:156-178, 211-231。

递归 V-cycle (1/2): 按 QUDA 的 residual 分支逐行展开

```
1 MG(level, x, b):
2     parity = target_parity_from_matpc_type()
3     outer = MAT_SOLUTION if b.is_full else MATPC_SOLUTION
4     inner = level.coarse_grid_solution_type
5     uniform = (level.smoother_solve_type == DIRECT_PC and inner == MATPC)
6               or (level.smoother_solve_type == DIRECT    and inner == MAT)
7     assert not (outer == MATPC and inner == MAT)
8     assert not (inner == MATPC and smoother_type != DIRECT_PC)
9
10    if level is coarsest:
11        (out, in) = dirac.prepare(x, b, outer)
12        if presmoothing exists: presmoothing(out, in)
13        dirac.reconstruct(x, b, outer)
14        return x
15
16    (out, in) = dirac.prepare(x, b, outer)
17    if presmoothing exists: presmoothing(out, in)
18    if not uniform: dirac.reconstruct(x, b, inner)
19
20    if presmoothing exists and uniform:
21        residual = presmoothing.returned_residual()
22    else if no presmoothing and uniform:
23        residual = in
```

递归 V-cycle (2/2): 限制、粗解、延拓、后平滑和最终重构

```
1  if transfer exists:
2      r_c = R(residual)
3      x_c = allocate_zero_coarse_like(r_c)
4      coarse_solver(x_c, r_c)          # recurse or bottom Krylov
5
6  if no presmoothing:
7      if inner == outer:
8          P_add(x, x_c)
9      else:
10         P_add(x[target_parity], x_c)
11  else:
12      correction = zero_like(r)
13      if inner == outer:
14          P(correction, x_c)
15          x += correction
16      else:
17          P(correction[target_parity], x_c)
18          x[target_parity] += correction[target_parity]
19
20  if not uniform:
21      (out, in) = dirac.prepare(x, b, inner)
22  if postsmoothing exists:
23      postsmoothing(out, in)          # use init guess = YES
```

V-cycle 的语义分层：平滑器、coarse solver 和 cycle wrapper 不是一回事

平滑器

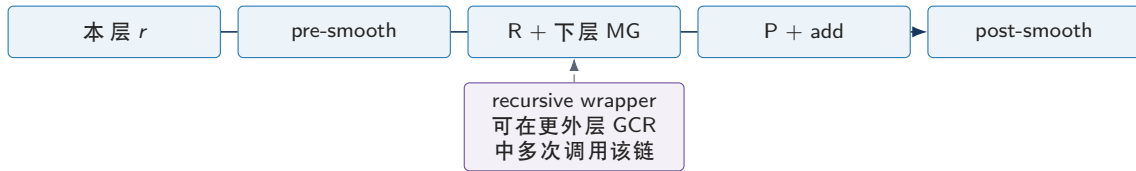
S_{pre} 和 S_{post} 是本层局部迭代； nu_{pre} 影响粗化前的误差， nu_{post} 清理粗校正带来的高频误差。

coarse solver

coarse_solver 接收 r_c ，输出 x_c 。非底层可以直接调用下层 MG；底层可以是 GCR/CG 等明确配置的 solver。

cycle type

VCYCLE 在允许的层级直接把下层 MG 作为 coarse solver；RECURSIVE 创建 PreconditionedSolver wrapper，把下层 MG 放进 Krylov 预条件链。



来源：cycle wrapper: lib/multigrid.cpp:535-702; 运行期 V-cycle: lib/multigrid.cpp:1171-1221。

Schwarz/MR 平滑器：每次 MR 更新都必须保留复数 和残差状态

$$\alpha_k = \frac{(Ar_k, r_k)}{(Ar_k, Ar_k)}, \quad x_{k+1} = x_k + \omega \alpha_k r_k, \quad r_{k+1} = r_k - \omega \alpha_k Ar_k.$$

```
1 MR_smooth(x, b, maxiter, omega, Nsteps, schwarz):
2     r = b - A*x if use_init_guess else b;  x = 0 if no_guess
3     for step in 0 .. Nsteps-1:
4         normalize = sqrt(initial_residual_norm2)
5         rs = r / normalize
6         k = 0
7         while k < maxiter and norm2(r) > delta2:
8             Ar = A_sloppy(rs)
9             alpha = dot(Ar, rs) / dot(Ar, Ar)  # complex
10            xs += omega * alpha * rs
11            rs -= omega * alpha * Ar
12            k += 1
13        x += normalize * xs
14        if compute_true_res or Nsteps > 1:
15            r = b - A*x
16        else:
17            r = normalize * rs
18        if step+1 == Nsteps or norm2(r) < stopping_threshold:
19            break
```

源码分析汇报 QUDA multigrid 复现手册

Schwarz 的作用：改变局部预条件器，不改变 MG 的 (R/P) 逻辑

Additive Schwarz

每次 active 块都用局部数据更新，两个颜色可同时准备，块间 hopping 暂时关闭；适合把局部块问题作为 MR 内部预条件。

Multiplicative Schwarz

red/black 交替更新，源码要求 Nsteps 为偶数；全局网格未做 block Schwarz 时还按 node parity 交替。

```
1 if block_schwarz:
2     if type == MULTIPLICATIVE:
3         active = RED if step even else BLACK
4         set_DD_active(active)
5     else:
6         # additive
7         set_DD_active(RED | BLACK,
8             no_block_hopping)
9         disable_global_reduction_if_blocks_local()
10
11 run_MR_inner_steps()
12
13 if block_schwarz:
14     reset_DD_masks()
```

setup 例外

普通 Krylov solver 不支持该 Schwarz 配置时，QUDA 在 null-vector finding 期间把

commDim 设为全 1、临时禁用 Schwarz，结

外层 GCR: 把一次 MG 应用当作可变预条件器 ($K(r)$)

```
1 flexible_GCR(A, b, K=MG, restart, maxiter):
2     x = initial_guess_or_zero(); r = b - A*x
3     basis_p = []; basis_Ap = []
4     while not converged(r) and iter < maxiter:
5         p = K(r)                                # one MG V-cycle / wrapper
6         Ap = A(p)
7         for j in 0 .. len(basis_Ap)-1:
8             beta = dot(basis_Ap[j], Ap) / dot(basis_Ap[j], basis_Ap[j])
9             Ap -= beta * basis_Ap[j]
10            p -= beta * basis_p[j]
11        alpha = dot(Ap, r) / dot(Ap, Ap)
12        x += alpha * p
13        r -= alpha * Ap
14        append(basis_p, p); append(basis_Ap, Ap)
15        if len(basis_p) == restart or reliable_update_needed(r):
16            r = b - A*x                            # true residual
17            basis_p.clear(); basis_Ap.clear()
18    return x
19
```

精度、MMA 和存储布局：只替换“如何算”，不替换“算什么”

precision_null	two-pass	MMA	halo precision
决定 (B/V) 及粗 link 的存储精度；CPU setup 小于单精度时源码提升到至少 single。	低精度 (V) 先求幅值、设 scale，再重算块正交；scale 必须随字段元数据传播。	setup/transfer/dslash 各自有开关；MMA 只改变 tile、order 和 kernel 调度，矩阵乘的共轭/转置语义不能变。	smoother halo 可独立于 (V/Y) 精度；setup 发现 quarter halo 时暂时提升到 half。



迁移顺序

先用 double/single CPU reference 固定数学结果，再逐项打开 parity、MPI、half/fixed-point、MMA；**每一步均回到 RP 、 $D_c - RDP$ 和 PC dslash 三个不变量。**

参数到行为映射 (1/2): 几何、零空间和 setup 参数

参数	所在阶段	实现行为与复现要求
n_level	层级	创建 $MG_0, \dots, MG_{n-1}$; 不得超过编译时最大递归深度。
n_vec	(B/V)	每层粗颜色列数; 必须不超过 aggregate 容量, 且决定粗算子矩阵大小。
n_vec_batch	null setup	每批同时生成的 B_i 数; $\text{len}(B) \% \text{batch} == 0$, 不整除直接失败。
geo_block_size	map/setup	逐维 b_μ ; 影响粗体积、局部容量、long-link 是否可安全粗化。
spin_block_size	map/setup	$s_f \rightarrow s_c$ 的整数分块; staggered 用 0 并映射成 2 个 coarse spin。
n_block_ortho	V	每个 aggregate 重复块 Gram-Schmidt 的次数。
block_ortho_two_pass	V 精度	half/fixed-point 下先量幅值再重新计算正交化。
precision_null	B/V/Y	null vector 和 coarse link 的存储/计算精度来源; CPU 小精度有提升规则。
setup_inv_type	B	CG/CA-CG 走 $D^\dagger D$, MG 走 GCR+MG 自举, 其他按 solver 工厂创建。
setup_maxiter, setup_tol	B	每次 setup solve 的最大迭代和相对残差目标; refresh 使用独立 maxiter。
num_setup_iter	B	每层重复“正交化 $\rightarrow$ 分批 solve $\rightarrow$ 正交化 $\rightarrow$ 重建/传播”的次数。

配置解读顺序

先检查 geo_block_size/spin_block_size/n_vec 的维数合法性, 再决定 setup solver 和精度; 顺序反过来会得到可运行但无法形成有效粗空间的配置。

来源: 参数声明: include/quda.h:638-701; 每层拷贝: include/multigrid.h:181-217; setup 使用: lib/multigrid.cpp:1279-1319, 1383-1413。

参数到行为映射 (2/2): 运行期、PC、cycle 和验证

参数	所在阶段	实现行为与复现要求
nu_pre / nu_post	V-cycle	分别创建 pre/post solver; pre 返回 residual, post 使用当前 $\times$ 作为初值。
smoother	smooth	MR/GCR/CA 等实际平滑器; MR 的 ω 更新与 Schwarz 需单独实现。
smoother_solve_type	prepare/粗层	DIRECT 或 DIRECT_PC; PC coarse solution 必须配 DIRECT_PC smoother。
coarse_grid_solution_type	R/P 接口	MAT 传 full field; MATPC 只传目标 parity, 并决定是否调用 prepare/reconstruct。
matpc_type	parity/Schur	由 EVEN_EVEN / ODD_ODD 选择目标 parity; PC 的 S_e/S_o 作用于 single-parity field。
cycle_type	coarse solver	VCYCLE 可直接链接下层 MG; RECURSIVE 通过 PreconditionedSolver 接到 Krylov。
use_init_guess	各 solver	null setup=YES; pre smoother=NO; post smoother=YES; coarse solver 默认 NO, deflation+coarse_guess 可打开。
run_verify	setup 后	检查 $PP^\dagger B$ 、 $P^\dagger P$ 、 $D_c - RDP$ 、PC dslash 和 normality。
generate_all_levels	setup 传播	YES: 下一层独立生成; NO: $B_{\ell+1} = RB_\ell$ 后重建 Transfer。
coarse_solver_*	粗解	给递归 wrapper 的 inverter、tol、maxiter、CA basis; 底层显式 Krylov 仍有自己的停止条件。
smoother_schwarz_*	smooth	additive/multiplicative、cycle 数、halo/reduction 行为; setup 普通 Krylov 期间可能被临时关闭。

从零实现的端到端伪代码：先单 rank，再逐层打开复杂度

```
1 main(cfg):
2     U, C = load_gauge_and_optional_clover(cfg)
3     D = make_fine_dirac(U, C, cfg.mass, cfg.kappa, cfg.mu)
4     levels = []
5     B = initialize_null_vectors(cfg.level[0])
6
7     for level in 0 .. cfg.n_level-1:
8         L = setup_level(level, D, B, cfg.level[level])
9         levels.append(L)
10        if level + 1 < cfg.n_level:
11            if cfg.generate_all_levels:
12                B = initialize_null_vectors(cfg.level[level+1])
13            else:
14                B = [L.R(v) for v in L.B]
15                D = L.coarse_operator()           # Y/X or Yhat/Xinv
16
17    assert verify_all_levels(levels, D, cfg)
18    x = zero_like(rhs)
19    for outer_iter in 1 .. cfg.outer_maxiter:
20        correction = MG(levels[0], zero_like(rhs), rhs)
21        update_outer_solution(x, correction, cfg.outer_solver)
22        if true_norm(rhs - D_fine(x)) <= cfg.outer_tol:
23            break
```

验证矩阵 (1/2): 把“构造正确”拆成五个可定位的误差源

编号	随机对象	计算	失败时优先定位
V1	细 B_i	$e_i = \ B_i - PRB_i\ /\ B_i\ $	map、spin map、块正交化、V layout
V2	粗 η	$e_c = \ \eta_c - RP\eta_c\ /\ \eta_c\ $	restrict 清零/累加、coarse parity、V 共轭
V3	粗随机 η	$e_D = \ (D_c - RDP)\eta\ /\ RDP\eta\ $	Y/X 分类、 γ 投影、 κ 、clover/mass
V4	单 parity ϕ	$e_{PC} = \ \hat{D} - A^{-1}D\ \phi/\ \cdot\ $	Xinv 逆、Yhat 左右乘、ghost 方向
V5	任意 η	$\text{Im}(\eta^\dagger M^\dagger M \eta)/\text{Re}(\cdot)$	dagger、spin/color 转置、parity 选择

精度相关阈值 (源码)

quarter/half: 5×10^{-2} ; single: 2×10^{-3} ; 更高精度默认 10^{-8} 。这是 QUDA verify 的检查阈值，不是所有物理配置的收敛保证。

来源：阈值与 V1/V2: lib/multigrid.cpp:749-880; V3/V4/V5 与 ASQTAD skip: lib/multigrid.cpp:883-1128。

特殊跳过

ASQTAD 长链接或过小 aggregate 会令 native coarse 与显式投影不完全对应；源码按 transfer type、operator 和 block size 条件跳过部分 verify，报告时必须保留该状态。

验证矩阵 (2/2): 每个测试都要记录范数、相对误差和配置

```
1 verify_level(L):
2     for i, B_i in enumerate(L.B):
3         coarse = L.R(B_i); projected = L.P(coarse)
4         report("null_span", i, norm2(B_i-projected)/norm2(B_i))
5
6     eta = random_coarse_field(L)
7     report("coarse_span", norm2(eta-L.R(L.P(eta)))/norm2(eta))
8
9     eta = random_coarse_field(L)
10    explicit = L.R(fine_D(L.P(eta)))
11    native = L.coarse_apply(eta)
12    report("galerkin", norm2(native-explicit)/norm2(explicit))
13
14    if L.has_pc:
15        phi = random_single_parity_field(L)
16        report("pc_dslash", norm2(L.pc_dslash(phi)-
17                                L.Xinv * L.unpreconditioned_dslash(phi)))
18
19    assert no_nan_or_inf(all_fields_and_scalars)
20
```

端到端复现阶梯：每一级都比上一级多一个可控变量



阶段	必须通过	不能提前声称
单 rank reference 单 GPU MPI	V1-V3、显式 coarse apply 与 RDP 一致 parity、field order、precision scale、V1-V5 单 rank 与多 rank 结果、ghost 边界、全局 reduction	GPU/MPI 性能 MPI overlap 任意 aggregate 跨 rank
多层/PC	每层 V1-V5、Schur prepare/reconstruct、外层真残差	只看 sloppy residual
参数扫描	固定 RHS/初值/硬件后比较 N_{vec}, b, ν	无基线的“加速”

停止条件

在当前层的所有不变量未通过前，不进入下一层。这样失败时能区分“数学构造错误”“通信错误”“精度错误”和“求解器参数错误”。

来源：动态测试入口可参考：tests/multigrid_evolve_test.cpp:103-179, 308-365、
tests/multigrid_benchmark_test.cpp:135-153； 本文为复现计划，非本次实测数据。
源码分析汇报 QUDA multigrid 复现手册

失败定位表：从哪一个不变量失败，就回到哪一段伪代码

症状	根因候选	最小定位实验
RP_η 偏离大 $B - PRB$ 偏离大	map 顺序、parity 或 restrict 未清零 块正交化列顺序、spin map、two-pass scale	关闭 GPU/MPI, 逐 aggregate 打印 $V^\dagger V - I$ $n_block_ortho=1$ 、double reference, 对比 每列归一范数
$D_c - RDP$ 偏离大 PC dslash 偏离大	internal/external 分类、 γ 符号、 κ 重复/遗漏 X^{-1} 左右乘反、backward 坐标错	用单一 point coarse basis, 逐方向比较 Y/X 只测一个 parity、一个方向, 打印 $Yhat_{fwd/back}$
单 rank 对但 MPI 错 残差下降后上升 setup 运行但粗解无效	ghost face index、parity pack、neighbor rank sloppy residual 当真 residual、GCR 更新/重构错 (B) 未按配置传播、nvec/batch 不匹配	只激活一个分区方向, 比较边界点逐项结果 每次 cycle 计算 (b-Ax), 禁用可靠更新优化 保存每层 (B,V), 逐层执行 V1-V3



来源：失败模式对应源码：lib/multigrid.cpp:745-1128、lib/multigrid.cpp:1131-1224、include/kernels/dslash_coarse.cuh:126-324；表为调试决策抽象。

源码控制流对照 (1/2): setup 与层级装配的逐段映射

复现伪代码阶段	QUDA 实现	读者应提取的规则
初始化/选择 (B)	lib/multigrid.cpp:36-62	random、file、eig、iterative、free-field 按条件分支; 不要混淆 compute_null_vector 与 vec_load。
创建 level	lib/multigrid.cpp:72-186	transfer/coarse fields/next MG/coarse solver 有严格依赖顺序。
创建 smoother	lib/multigrid.cpp:273-337	pre 返回 residual、post 使用 init guess; 精度、Schwarz、tol 在此装配。
创建 coarse Dirac	lib/multigrid.cpp:342-434	根据 coarse_grid_solution_type 与 smoother solve type 选择 coarse 或 coarse-PC。
setup solver	lib/multigrid.cpp:1275-1363	maxiter/tol/use-init/batch/CG vs GCR vs MG 自举。
batch + 正交	lib/multigrid.cpp:1365-1427	pre/post global orthonormalize、TEST_VECTOR 分支、每批写回 (B)。
向下传播	lib/multigrid.cpp:1429-1451	generate-all 或 (R B), 随后 reset 下一层。

如何使用此表

先按左列实现与测试, 再按中列检查 QUDA 特有的状态更新; 右列是不能从 “V-cycle” 三个字推出来、但会影响复现的行为。

来源: 以上全部为当前快照的直接源码行号; 伪代码并非源码直引, 而是按控制流重写。

源码控制流对照 (2/2): transfer、Galerkin、运行期和验证

复现伪代码阶段	QUDA 实现	对齐点
geometry map	lib/transfer.cpp:200-241	fine-to-coarse 逐点除 block; coarse-to-fine 按 coarse index 排序。
spin map	lib/transfer.cpp:243-257	non-staggered 为 s/b_s , staggered parity 映射到 0/1。
P/R dispatch	lib/transfer.cpp:259-328	aggregate、optimized KD、coarse KD 是不同 transfer; 本报告主线是 aggregate。
block ortho	include/kernels/block_orthogonalize.cuh:141-253	aggregate 内 dot/reduce/subtract/normalize, 支持重复和 tile。
UV/VUV	lib/coarse_op.cuh:1043-1457	nFace、双向、long link、diagonal split、identity/mass/twist/convert。
coarse apply	include/kernels/dslash_coarse.cuh:126-324	forward/backward local/halo gather, local X , $-\kappa$ 汇总。
Yhat/Schur	lib/dirac_coarse.cpp:506-649	X^{-1} 、 $\hat{Y}$ 、M、prepare、reconstruct、向下粗化 $\hat{Y}$ 。
V-cycle	lib/multigrid.cpp:1131-1224	residual 来源和 solution type 分支。
verify	lib/multigrid.cpp:745-1128	阈值、五类检查及 ASQTAD skip 条件。

最终原则

先复现左列的数学动作，再逐项加入右列的布局/通信优化；不把某个 CUDA kernel 的线程映射误认为 multigrid 算法本
源码分析汇报 QUDA multigrid 复现手册 57/88

附录 A1: 运行期先确定 solution type (源码直引)

```
1   if (param.level < param.Nlevel - 1) {  
2       std::vector<ColorSpinorField> out(b.size()), in(b.size());  
3       diracSmoother->prepare(out, in, x, b, outer_solution_type);  
4  
5       if (presmoother) (*presmoother)(out, in);  
6  
7       if (!smoother_solver_uniform) diracSmoother->reconstruct(x, b, inner_solution_type);  
8  
9       if (compute_residual) {  
10         (*param.matResidual)(r, x);  
11     }
```

对应伪代码

这是非底层分支的入口：创建工作字段，执行 prepare，再决定是否预平滑。

来源：原文件：lib/multigrid.cpp:1171-1180；代码未转写。

附录 A2: 显式 residual 分支 (源码直引)

```
1     axpby(1.0, b, -1.0, r);
2 }
3
4 // We need this to ensure that the coarse level has been created.
5 // e.g. in case of iterative setup with MG we use just pre- and post-smoothing at the first
iteration.
6 if (transfer) {
7     const auto &residual = use_solver_residual ? presmoothing->get_residual() :
8         !presmoothing && smoother_solver_uniform ? in :
9         !presmoothing ? b :
10         r;
```

对应伪代码

只有不一致的 smoother/coarse solution type 才显式重算 $r = b - Ax$; 其余路径复用 solver residual 或输入。

来源: 原文件: lib/multigrid.cpp:1181-1190; 代码未转写。

附录 A3: 限制、粗解与回传 (源码直引)

```
1      // restrict to the coarse grid
2      transfer->R(r_coarse, residual);
3
4
5      // recurse to the next lower level
6      (*coarse_solver)(x_coarse, r_coarse);
7
8      // prolongate back to this grid
9      if (!presmoothen) {
10         transfer->P(inner_solution_type == outer_solution_type ? x : x(parity), x_coarse);
```

对应伪代码

R 把本层 residual 送到下一层; 粗解返回后按 full 或目标 parity 调用 P。

来源: 原文件: lib/multigrid.cpp:1191-1200; 代码未转写。

附录 A4: 有预平滑时必须做 correction add (源码直引)

```
1      } else { // we must sum to the presmoother solution
2          auto res = inner_solution_type == outer_solution_type ? cvector_ref<ColorSpinorField>(r)
3              :
4              cvector_ref<ColorSpinorField>(r)(
5              parity);
6          transfer->P(res, x_coarse);
7          xpy(res, inner_solution_type == outer_solution_type ? x : x(parity));
8      }
9      if (!smoother_solver_uniform) diracSmoother->prepare(out, in, x, b, inner_solution_type);
```

不可替换语义

有 pre-smoother 时, P 的输出写到临时 correction, 再执行 $x \leftarrow x + P x_c$; 直接覆盖 x 会丢失平滑结果。

来源: 原文件: lib/multigrid.cpp:1201-1210; 代码未转写。

附录 A5: post smoother 与 coarsest-level 分支 (源码直引)

```
1     if (postsmoother) (*postsmoother)(out, in);
2
3     diracSmoother->reconstruct(x, b, outer_solution_type);
4
5 } else { // do the coarse grid solve
6
7     std::vector<ColorSpinorField> out(b.size()), in(b.size());
8     diracSmoother->prepare(out, in, x, b, outer_solution_type);
9     if (presmoother) (*presmoother)(out, in);
10    diracSmoother->reconstruct(x, b, outer_solution_type);
11 }
```

对应伪代码

回到本层后重新 prepare (若 solution type 改变), 执行 post, 再以 outer solution type reconstruct。

来源: 原文件: lib/multigrid.cpp:1211-1221; 代码未转写。

附录 B1: 块正交化的循环边界 (源码直引)

```
1 // loop over number of block orthos
2 for (int n = 0; n < arg.nBlockOrtho; n++) {
3     for (int j = 0; j < Arg::nVec; j += mVec) {
4
5         ColorSpinor<real, nColor, spinBlock> v[mVec][n_sites_per_thread];
6
7         for (int tx = 0; tx < n_sites_per_thread; tx++) {
8             if (x_offset_cb[tx] >= arg.aggregate_size_cb) break;
9             if (n == 0) { // load from B on first Gram-Schmidt, otherwise V.
10                 if (chirality == 0) {
11 #pragma unroll
12                 for (int m = 0; m < mVec; m++) arg.B[j+m].template load<spinBlock>(v[m][tx].data,
                    parity[tx], x_cb[tx], 0);
```

对应伪代码

外层是重复次数, 内层按 $mVec$ 分批; 第一次从 B 读, 后续从已写回的 V 读。

来源: 原文件: `include/kernels/block_orthogonalize.cuh:170-181`; 代码未转写。

附录 B2: 按 chirality 读取候选列 (源码直引)

```
1         } else {
2 #pragma unroll
3         for (int m = 0; m < mVec; m++) arg.B[j+m].template load<spinBlock>(v[m][tx].data,
4         parity[tx], x_cb[tx], 1);
5         }
6     } else {
7 #pragma unroll
8     for (int m = 0; m < mVec; m++) load(v[m][tx], parity[tx], x_cb[tx], chirality, j + m)
9     ;
10    }
11 }
12
13 for (int i = 0; i < j; i++) { // compute (j,i) block inner products
14     ColorSpinor<real, nColor, spinBlock> vi[n_sites_per_thread];
```

对应伪代码

奇偶与 chirality 的索引在装载 B_i 时已经决定; 这就是 spin map 不能在 P/R 阶段另起一套的原因。

附录 B3: 跨历史列的块内积 (源码直引)

```
1      dot_t dot{0};
2      for (int tx = 0; tx < n_sites_per_thread; tx++) {
3          if (x_offset_cb[tx] >= arg.aggregate_size_cb) break;
4          load(vi[tx], parity[tx], x_cb[tx], chirality, i);
5
6      #pragma unroll
7          for (int m = 0; m < mVec; m++) dot[m] += innerProduct(vi[tx], v[m][tx]);
8      }
9
10     dot = dot_reducer.template AllSum<false>(dot);
11
```

对应伪代码

每个历史列 $i < j$ 都先做 aggregate reduction; 得到的复系数再用于整个当前 tile 的减法。

来源: 原文件: include/kernels/block_orthogonalize.cuh:194-205; 代码未转写。

附录 B4: 减去历史列并进入 tile 正交 (源码直引)

```
1      // subtract the blocks to orthogonalise
2      for (int tx = 0; tx < n_sites_per_thread; tx++) {
3          if (x_offset_cb[tx] >= arg.aggregate_size_cb) break;
4      #pragma unroll
5          for (int m = 0; m < mVec; m++)
6              v[m][tx] = caxpy(-complex<real>(dot[m].real(), dot[m].imag()), vi[tx], v[m][tx]);
7      }
8  } // i
9
10     // now orthogonalize over the block diagonal and normalize each entry
11 #pragma unroll
12     for (int m = 0; m < mVec; m++) {
```

顺序

先减去 $i < j$ 的历史列, 再对 tile 内 m 列做列间正交; 两层循环不能合并成一次局部归一。

来源: 原文件: include/kernels/block_orthogonalize.cuh:206-217; 代码未转写。

附录 B5: tile 内列间内积 (源码直引)

```
1      dot_t dot{0};
2      for (int tx = 0; tx < n_sites_per_thread; tx++) {
3          if (x_offset_cb[tx] >= arg.aggregate_size_cb) break;
4      #pragma unroll
5          for (int i = 0; i < m; i++) dot[i] += innerProduct(v[i][tx], v[m][tx]);
6      }
7
8      dot = dot_reducer.template AllSum<false>(dot);
9
10
11      sum_t nrm = 0.0;
12      for (int tx = 0; tx < n_sites_per_thread; tx++) {
```

对应伪代码

对当前列 m , 只与 $i < m$ 的当前 tile 列做内积, 并把所有线程贡献归并到 aggregate 范围。

来源: 原文件: include/kernels/block_orthogonalize.cuh:218-229; 代码未转写。

附录 B6: tile 内减法与归一化系数 (源码直引)

```
1         if (x_offset_cb[tx] >= arg.aggregate_size_cb) break;
2     #pragma unroll
3         for (int i = 0; i < m; i++)
4             v[m][tx] = caxpy(-complex<real>(dot[i].real(), dot[i].imag()), v[i][tx],
5                             v[m][tx]); // subtract the blocks to orthogonalise
6         nrm += norm2(v[m][tx]);
7     }
8
9     nrm = norm_reducer.template AllSum<false>(nrm);
10    auto nrm_inv = nrm > 0.0 ? quda::rsqrt(nrm) : 0.0;
11
12    for (int tx = 0; tx < n_sites_per_thread; tx++) {
```

对应伪代码

先按 $i < m$ 更新 v_m , 再用 aggregate norm 求 n_m^{-1} ; 零范数由实现保留为零系数并应在 reference 中报错。

来源: 原文件: include/kernels/block_orthogonalize.cuh:230-241; 代码未转写。

附录 B7: 归一化写回与重复循环结束 (源码直引)

```
1         if (x_offset_cb[tx] >= arg.aggregate_size_cb) break;
2         v[m][tx] *= nrm_inv;
3     }
4 }
5
6     for (int tx = 0; tx < n_sites_per_thread; tx++) {
7         if (x_offset_cb[tx] >= arg.aggregate_size_cb) break;
8 #pragma unroll
9         for (int m = 0; m < mVec; m++) save(parity[tx], x_cb[tx], chirality, j + m, v[m][tx]);
10    }
11 } // j
12 } // n
```

实现边界

归一化后的 tile 才写入 V ; 下一批次和下一次重复都读取这个持久结果。

来源: 原文件: `include/kernels/block_orthogonalize.cuh:242-253`; 代码未转写。

附录 B8: VUV 的坐标与 aggregate 判定 (源码直引)

```
1  template <int nFace, typename Ftor>
2  __device__ __host__ void computeVUV(const Ftor &ftor, int parity, int x_cb, int i0, int j0, int
   parity_coarse_,
3                                     int coarse_x_cb_)
4  {
5      using Arg = typename Ftor::Arg;
6      const Arg &arg = ftor.arg;
7      using real = typename Arg::Float;
8      constexpr int nDim = 4;
9      int coord[QUDA_MAX_DIM];
10     int coord_coarse[QUDA_MAX_DIM];
11
12     getCoords(coord, x_cb, arg.x_size, parity);
```

对应伪代码

先从 checkerboard 坐标恢复 coarse 坐标, 再用 fine hop 的终点判断该矩阵块属于 X 还是 Y。

来源: 原文件: include/kernels/coarse_op_kernel.cuh:1608-1619; 代码未转写。

附录 B9: VUV 的 diagonal 判定与 coarse parity (源码直引)

```
1   for (int d = 0; d < nDim; d++) coord_coarse[d] = coord[d]/arg.geo_bs[d];
2
3   //Check to see if we are on the edge of a block.  If adjacent site
4   //is in same block, M = X, else M = Y
5   const bool isFromCoarseClover = Arg::fineSpin == 2 && arg.dir == QUDA_IN_PLACE;
6   const bool isDiagonal = (isFromCoarseClover || isCoarseDiagonal(coord, coord_coarse, arg.dim,
7   nFace, arg)) ? true : false;
8
9   int coarse_parity = arg.shared_atomic ? parity_coarse_ : 0;
10  if (!arg.shared_atomic) {
11      for (int d=0; d<nDim; d++) coarse_parity += coord_coarse[d];
12      coarse_parity &= 1;
13      coord_coarse[0] /= 2;
```

对应伪代码

isDiagonal 决定 X/Y, coarse parity 与 checkerboard index 决定矩阵写回位置; 这两个判断不能互换。

来源: 原文件: include/kernels/coarse_op_kernel.cuh:1620-1631; 代码未转写。

附录 B10: VUV tile 乘法与局部系数 (源码直引)

```
1     }
2     int coarse_x_cb = arg.shared_atomic ? coarse_x_cb_ : ((coord_coarse[3]*arg.xc_size[2]+
   coord_coarse[2])*arg.xc_size[1]+coord_coarse[1])*(arg.xc_size[0]/2) + coord_coarse[0];
3
4     using Ctype = decltype(make_tile_C<complex<real>, false>(arg.vuvTile));
5     Ctype vuv[Arg::coarseSpin * Arg::coarseSpin];
6     multiplyVUV(vuv, arg, parity, x_cb, i0, j0);
7
8     if (isDiagonal && !isFromCoarseClover) {
9 #pragma unroll
10     for (int s2=0; s2<Arg::coarseSpin*Arg::coarseSpin; s2++) vuv[s2] *= -arg.kappa;
11     }
```

系数检查

VUV 先形成 coarse spin block; diagonal hopping 再乘 $-\kappa$, 而 local X 的其他项另由专门 kernel 加入。

来源: 原文件: include/kernels/coarse_op_kernel.cuh:1632-1643; 代码未转写。

附录 B11: VUV 的 atomic 写回 (源码直引)

```
1     if (arg.shared_atomic)
2         storeCoarseSharedAtomic(vuv, isDiagonal, coarse_x_cb, coarse_parity, i0, j0, parity, ftor);
3     else
4         storeCoarseGlobalAtomic(vuv, isDiagonal, coarse_x_cb, coarse_parity, i0, j0, arg);
```

对应伪代码

最后依据共享或全局 atomic 路径写回；线程映射可以改变， X/Y 分类和矩阵转置语义不能改变。

来源：原文件：include/kernels/coarse_op_kernel.cuh:1644-1647；代码未转写。

附录 C1: Yhat 先处理 backward link (源码直引)

```
1 // first do the backwards links  $Y^{+\mu} * X^{-\dagger}$ 
2 if (arg.comm_dim[d] && is_boundary(coord, d, 0, arg)) {
3
4     auto yHat = make_tile_C<complex,true>(arg.tile);
5
6     #pragma unroll
7     for (int k = 0; k < Arg::yhatTileType::k; k += Arg::yhatTileType::K) {
8         auto Y = make_tile_A<complex, true>(arg.tile);
9         Y.load(arg.Y, d, 1-parity, ghost_idx, i0, k);
10
11         auto X = make_tile_Bt<complex, false>(arg.tile);
12         X.load(arg.Xinv, 0, parity, x_cb, j0, k);
```

对应伪代码

边界使用 ghost index, 内部使用 hop index; 两条路径都加载同一个目标点的 $X^{-\dagger}$ 。

来源: 原文件: include/kernels/coarse_op_preconditioned.cuh:60-71; 代码未转写。

附录 C2: Yhat backward 的矩阵乘与存储 (源码直引)

```
1      yHat.mma_nt(Y, X);
2  }
3
4
5  if constexpr (Arg::compute_max) {
6      yHatMax = yHat.abs_max();
7  } else {
8      yHat.save(arg.Yhat, d, 1 - parity, ghost_idx, i0, j0);
9  }
10
11 } else {
12     const int back_idx = linkIndexHop(coord, arg.dim, d, -arg.nFace);
```

方向

backward 的 Y 在邻点位置读取, 并以 $X^{-\dagger}$ 右乘; 这正是 PC link 最容易写反的地方。

来源: 原文件: `include/kernels/coarse_op_preconditioned.cuh:72-83`; 代码未转写。

附录 C3: Yhat 内部 backward 路径 (源码直引)

```
1      auto yHat = make_tile_C<complex,false>(arg.tile);
2
3
4  #pragma unroll
5      for (int k = 0; k < Arg::yhatTileType::k; k += Arg::yhatTileType::K) {
6          auto Y = make_tile_A<complex, false>(arg.tile);
7          Y.load(arg.Y, d, 1-parity, back_idx, i0, k);
8
9          auto X = make_tile_Bt<complex, false>(arg.tile);
10         X.load(arg.Xinv, 0, parity, x_cb, j0, k);
11
12         yHat.mma_nt(Y, X);
```

对应伪代码

非边界使用 back_idx; 矩阵 tile 的 B^T 装载实现右乘, 不能替换为普通左乘。

来源: 原文件: include/kernels/coarse_op_preconditioned.cuh:84-95; 代码未转写。

附录 C4: Yhat backward 的输出 (源码直引)

```
1     }  
2     if constexpr (Arg::compute_max) {  
3         yHatMax = yHat.abs_max();  
4     } else {  
5         yHat.save(arg.Yhat, d, 1 - parity, back_idx, i0, j0);  
6     }  
7 }  
8  
9 { // now do the forwards links  $X^{-1} * Y^{-\mu}$   
10     auto yHat = make_tile_C<complex, false>(arg.tile);  
11  
12     #pragma unroll
```

对应伪代码

backward 结果保存到反向方向的 link; 随后进入 forward 的 $X^{-1}Y$ 路径。

来源: 原文件: include/kernels/coarse_op_preconditioned.cuh:96-107; 代码未转写。

附录 C5: Yhat forward 是左乘 (源码直引)

```
1   for (int k = 0; k < Arg::yhatTileType::k; k += Arg::yhatTileType::K) {  
2       auto X = make_tile_A<complex, false>(arg.tile);  
3       X.load(arg.Xinv, 0, parity, x_cb, i0, k);  
4  
5       auto Y = make_tile_B<complex, false>(arg.tile);  
6       Y.load(arg.Y, d + 4, parity, x_cb, k, j0);  
7  
8       yHat.mma_nn(X, Y);  
9   }  
10  if constexpr (Arg::compute_max) {  
11      yHatMax = max(yHatMax, yHat.abs_max());  
12  } else {
```

方向

forward 先加载 $X^{-1}(q)$, 再加载 $Y_{+\mu}(q)$, 计算 $X^{-1}Y$ 并保存到正向方向。

来源: 原文件: include/kernels/coarse_op_preconditioned.cuh:108-119; 代码未转写。

附录 C6: Yhat forward 写回 (源码直引)

```
1     yHat.save(arg.Yhat, d + 4, parity, x_cb, i0, j0);  
2 }
```

对应伪代码

两个方向的 ghost/内部索引都在本 kernel 内闭合；外部实现只需保持相同的矩阵方向和字段布局。

来源：原文件：include/kernels/coarse_op_preconditioned.cuh:120-121；代码未转写。

附录 D1: coarse dslash 的 forward gather (源码直引)

```
1   if (!thread_dir || target::is_host()) {
2
3       //Forward gather - compute fwd offset for spinor fetch
4   #pragma unroll
5       for(int d0 = 0; d0 < Arg::nDim; d0 += Arg::dim_stride) { // loop over dimension
6           int d = d0 + thread_dim;
7           const int fwd_idx = linkIndexHop(coord, arg.dim, d, arg.nFace);
8
9           if (arg.commDim[d] && is_boundary(coord, d, 1, arg) ) {
10               if constexpr (doHalo<Arg::type>()) {
11                   int ghost_idx = ghostFaceIndex<1>(coord, arg.dim, d, arg.nFace);
```

对应伪代码

forward 访问 $q + \hat{\mu}$; 跨 rank 时把本地 index 换成 ghost index, 目标 parity 保持不变。

来源: 原文件: include/kernels/dslash_coarse.cuh:148-159; 代码未转写。

附录 D2: forward halo 中的矩阵乘 (源码直引)

```
1 #pragma unroll
2     for(int color_local = 0; color_local < Mc; color_local++) { //Color row
3         int c_row = color_block + color_local; // global color index
4         int row = s_row * Arg::nColor + c_row;
5 #pragma unroll
6         for(int s_col = 0; s_col < Arg::nSpin; s_col++) { //Spin column
7 #pragma unroll
8             for(int c_col = 0; c_col < Arg::nColor; c_col += Arg::color_stride) { //Color
9                 column
10                    int col = s_col * Arg::nColor + c_col + color_offset;
11                    if (!Arg::dagger)
12                        out[color_local] = cmac(arg.Y(d+4, parity, x_cb, row, col),
13                                                arg.halo.Ghost(d, 1, their_spinor_parity, ghost_idx +
14                                                            src_idx * arg.ghostFaceCB[d], s_col, c_col+color_offset), out[color_local]);
```

对应伪代码

ghost spinor 按 source batch、spin、color 索引; $Y_{+\mu}$ 作用在输出 spin/color 行上。

来源: 原文件: include/kernels/dslash_coarse.cuh:160-171; 代码未转写。

附录 D3: forward bulk 与 dagger 分支 (源码直引)

```
1         else
2             out[color_local] = cmac(arg.Y(d, parity, x_cb, row, col),
3                                     arg.halo.Ghost(d, 1, their_spinor_parity, ghost_idx +
4                                     src_idx * arg.ghostFaceCB[d], s_col, c_col+color_offset), out[color_local]);
5         }
6     }
7 }
8 } else if constexpr (doBulk<Arg::type>()) {
9 #pragma unroll
10     for(int color_local = 0; color_local < Mc; color_local++) { //Color row
11         int c_row = color_block + color_local; // global color index
12         int row = s_row * Arg::nColor + c_row;
```

核对点

dagger 模式改变读取的方向 link; 非 dagger 模式读取 $Y[d+4]$ 。这不是简单地给最终向量取共轭。

来源: 原文件: include/kernels/dslash_coarse.cuh:172-183; 代码未转写。

附录 D4: forward bulk 读取 (源码直引)

```
1 #pragma unroll
2     for(int s_col = 0; s_col < Arg::nSpin; s_col++) { //Spin column
3 #pragma unroll
4     for(int c_col = 0; c_col < Arg::nColor; c_col += Arg::color_stride) { //Color column
5         int col = s_col * Arg::nColor + c_col + color_offset;
6         if (!Arg::dagger)
7             out[color_local] = cmac(arg.Y(d+4, parity, x_cb, row, col),
8                                     arg.inA[src_idx](their_spinor_parity, fwd_idx, s_col,
9 c_col+color_offset), out[color_local]);
10        else
11            out[color_local] = cmac(arg.Y(d, parity, x_cb, row, col),
12                                    arg.inA[src_idx](their_spinor_parity, fwd_idx, s_col,
13 c_col+color_offset), out[color_local]);
14    }
```

对应伪代码

内部点直接用 `fwd_idx` 读取输入；通信和计算的数学结果必须逐点一致。

来源：原文件：include/kernels/dslash_coarse.cuh:184-195；代码未转写。

附录 D5: 切换到 backward gather (源码直引)

```
1         }
2     }
3 }
4
5     } // nDim
6 }
7
8     if (thread_dir || target::is_host()) {
9
10         //Backward gather - compute back offset for spinor and gauge fetch
11         #pragma unroll
12         for(int d0 = 0; d0 < Arg::nDim; d0 += Arg::dim_stride) {
```

对应伪代码

backward 使用 $q - \hat{\mu}$ 的 link 和输入; GPU 中 forward/backward 可由不同线程负责, 但归约顺序需固定。

来源: 原文件: include/kernels/dslash_coarse.cuh:196-207; 代码未转写。

附录 D6: backward 的 ghost 分支 (源码直引)

```
1      const int d = d0 + thread_dim;
2      const int back_idx = linkIndexHop(coord, arg.dim, d, -arg.nFace);
3
4      if (arg.commDim[d] && is_boundary(coord, d, 0, arg)) {
5          if constexpr (doHalo<Arg::type>()) {
6              const int ghost_idx = ghostFaceIndex<0>(coord, arg.dim, d, arg.nFace);
7  #pragma unroll
8              for (int color_local=0; color_local<Mc; color_local++) {
9                  int c_row = color_block + color_local;
10                 int row = s_row * Arg::nColor + c_row;
11  #pragma unroll
12                 for (int s_col=0; s_col < Arg::nSpin; s_col++)
```

边界

backward 边界先用 `ghostFaceIndex` 定位 link 与 spinor; 两者的 face 顺序必须使用同一 parity 约定。

来源: 原文件: `include/kernels/dslash_coarse.cuh:208-219`; 代码未转写。

附录 D7: backward link 的共轭转置 (源码直引)

```
1 #pragma unroll
2     for (int c_col=0; c_col < Arg::nColor; c_col += Arg::color_stride) {
3         int col = s_col * Arg::nColor + c_col + color_offset;
4         if (!Arg::dagger)
5             out[color_local] = cmac(conj(arg.Y.Ghost(d, 1-parity, ghost_idx, col, row)),
6                                     arg.halo.Ghost(d, 0, their_spinor_parity, ghost_idx +
7 src_idx * arg.ghostFaceCB[d], s_col, c_col+color_offset), out[color_local]);
8         else
9             out[color_local] = cmac(conj(arg.Y.Ghost(d+4, 1-parity, ghost_idx, col, row)),
10                                     arg.halo.Ghost(d, 0, their_spinor_parity, ghost_idx +
11 src_idx * arg.ghostFaceCB[d], s_col, c_col+color_offset), out[color_local]);
12     }
```

对应伪代码

非 dagger 路径使用 $Y_d^\dagger(q - \hat{\mu})$; 这一共轭转置属于 link, 不属于输入 spinor。

来源: 原文件: include/kernels/dslash_coarse.cuh:220-231; 代码未转写。

附录 D8: backward bulk 与局部 link 索引 (源码直引)

```
1      } else if constexpr (doBulk<Arg::type>()) {  
2          const int gauge_idx = back_idx;  
3          #pragma unroll  
4          for(int color_local = 0; color_local < Mc; color_local++) {  
5              int c_row = color_block + color_local;  
6              int row = s_row * Arg::nColor + c_row;  
7          #pragma unroll  
8          for(int s_col = 0; s_col < Arg::nSpin; s_col++)  
9          #pragma unroll  
10             for(int c_col = 0; c_col < Arg::nColor; c_col += Arg::color_stride) {  
11                 int col = s_col * Arg::nColor + c_col + color_offset;  
12                 if (!Arg::dagger)
```

对应伪代码

bulk 路径使用 back_idx; Y 的 parity 是邻点 parity, 而输出仍属于当前目标 parity。

来源: 原文件: include/kernels/dslash_coarse.cuh:232-243; 代码未转写。

附录 D9: local X 与 $-\kappa$ 汇总 (源码直引)

```
1         out[color_local] = cmac(conj(arg.Y(d, 1-parity, gauge_idx, col, row)),
2                                 arg.inA[src_idx](their_spinor_parity, back_idx, s_col,
3         c_col+color_offset), out[color_local]);
4         else
5             out[color_local] = cmac(conj(arg.Y(d+4, 1-parity, gauge_idx, col, row)),
6                                     arg.inA[src_idx](their_spinor_parity, back_idx, s_col,
7         c_col+color_offset), out[color_local]);
8     }
```

对应伪代码

hopping 汇总后才统一乘 $-\kappa$; X 作为 local matrix 另行加入, 故装配时不能把该系数重复乘进 Y 。

来源: 原文件: include/kernels/dslash_coarse.cuh:244-249; 代码未转写。

附录 D：建议的最小配置与报告记录模板

```
1 geometry: Lf[4], periodic/BC
2 operator: Wilson or Clover, kappa, mass
3 level[0]: b[4], spin_block, nvec
4 setup: num_setup_iter, maxiter, tol
5 ortho: global_pre/post, n_block_ortho
6 solve: smoother, nu_pre, nu_post, omega
7 coarse: solution_type, solve_type, cycle
8 precision: B/V/Y/X/halo
9 parallel: nranks, rank_grid, face depth
10 verify: V1..V5 values and thresholds
11
```

每次回归至少记录

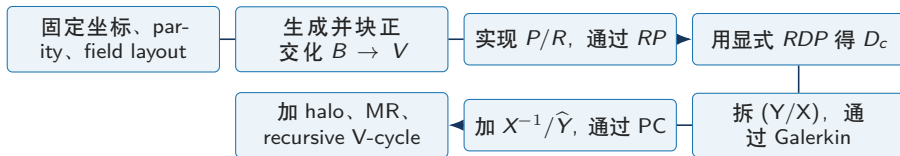
字段	内容
输入	gauge/clover 来源、随机种子、边界
结构	每层 (L,b,Nvec,Nspin,parity)
算法	setup type、smoother、cycle、PC
误差	V1-V5 的 L2/max、真 residual
性能	设备、rank、时间、基线（若已测）

真实性

没有动态命令和输出时，性能/收敛栏写“未验证”，不能用源码行号代替运行证据。

来源：参数字段：include/quda.h:638-835；测试参数入口：tests/utils/set_params.cpp:361-627。本页是复现实验记录模板。

最终结论：复现 QUDA multigrid 的最短正确路径



已完成

本版补齐语言无关 setup、null-vector batch、两级块正交、 P/R 、Galerkin、PC Schur、halo、MR、GCR、参数映射和复现阶梯。

来源：总结依据：lib/multigrid.cpp:1131-1224、lib/transfer.cpp:200-328、lib/dirac_coarse.cpp:121-170, 506-649；结论中的动态缺口为本次工作边界。

可证据化

每个关键步骤均有 QUDA 文件：行号；附录保留运行期、正交化、VUV、Yhat、dslash 直引片段。

尚待动态验证

单 GPU/MPI 实际残差、不同参数的收敛曲线和硬件性能；应按“单 rank reference→GPU→MPI→多层”阶梯补测。